DESIGN AND DEVELOPMENT OF A STATISTICAL MEASURES SIMULATOR

An Interactive Web-Based Tool for Grouped and Ungrouped Data

Anwesha Bag (10271025006) **Poulami Laha** (10271025021) **Shreya Mandal** (10271025037)

Department of Computer Application, Kalyani Government Engineering College
MCA, Semester III | Odd Semester, 2026–27

Abstract

This paper presents the design and implementation of a browser-based Statistical Measures Simulator for calculating the mean, median, mode and population standard deviation from both ungrouped and grouped data. The application provides separate data-entry interfaces, supports dynamic row addition and removal, includes sample datasets, validates user input, and displays calculated values together with intermediate calculation details. HTML is used to structure the interface, CSS to style the application and support responsive presentation, and JavaScript to perform the statistical operations. For ungrouped observations, the measures are calculated directly from the entered values. For grouped frequency distributions, class midpoints and standard grouped-data formulae are used, so the resulting statistics are estimates. The simulator is intended as an educational aid for understanding descriptive statistics and the computational steps behind them.

Keywords: Descriptive statistics, mean, median, mode, standard deviation, grouped data, ungrouped data, HTML, CSS, JavaScript.

1. Introduction

Descriptive statistics provides concise numerical summaries of a dataset. Measures of central tendency — mean, median and mode — describe different aspects of the centre of a distribution, while standard deviation describes the spread of observations around the mean. These measures help transform a list of raw values into information that can be interpreted and compared. The mean uses every observation, the median identifies the central value after ordering the data, and the mode identifies the most frequent value. Standard deviation expresses variability in the same units as the data. [1]

In practice, data may be available as individual observations (ungrouped data) or summarized into class intervals with corresponding frequencies (grouped data). Exact calculations are possible when the original observations are available. When only grouped intervals are known, the individual values are unavailable; therefore, calculations use class midpoints and interpolation-based formulae to produce estimates. [1] This project develops an interactive simulator that demonstrates both cases within a single web interface.

The objective is to provide a simple tool through which learners can enter data, calculate the four statistical measures, inspect the calculation details, and observe how the procedures differ between grouped and ungrouped datasets.

2. Objectives

The main objectives of the project are: (i) to design an accessible web interface for entering ungrouped observations and grouped class intervals; (ii) to calculate mean, median, mode and population standard deviation; (iii) to validate incomplete or invalid entries and overlapping class intervals; (iv) to display intermediate values and frequency tables; and (v) to provide sample datasets for demonstration and practice.

3. Methodology and System Design

The simulator is implemented using three client-side technologies. HTML defines the page structure, including tabs, input tables, buttons and the results section. CSS controls typography, colours, spacing, cards, tables and the responsive layout. JavaScript manages row creation, tab switching, sample-data loading, input validation, statistical calculations and result presentation.

3.1 Ungrouped-data procedure

The user enters numerical observations in a dynamically generated table. The program checks that at least one value is present and that every entry is a finite number. It then computes the number of observations n and the sum of values. The mean is Σx/n. The values are sorted in ascending order; the median is the middle value for odd n and the average of the two middle values for even n. A frequency map is used to identify the value or values with the highest frequency. If every value occurs only once, the program reports "No mode". Population variance is calculated as $\Sigma(x - \text{mean})^2/n$ and the population standard deviation is its square root.

3.2 Grouped-data procedure

Each row contains a lower class limit, upper class limit and frequency. Midpoint: m = (lower limit + upper limit)/2. The program validates entries, sorts classes and rejects overlapping intervals. Total frequency: N = sum(f). Estimated mean: x_bar = sum(f*m)/N. Population variance: variance = sum[f*(m - x_bar)^2]/N.
Median = L + [(N/2 - cf)/f] * h
Here, L is the lower limit of the median class, cf is the cumulative frequency before it, f is its frequency and h is its class width.
Mode = L + [(f_m - f_1)/(2*f_m - f_1 - f_2)] * h
Here, L and h refer to the modal class; f_m is its frequency, f_1 is the preceding-class frequency and f_2 is the succeeding-class frequency. The modal class has the highest frequency.

3.3 Validation and output

The application displays a readable error message when required values are missing, numeric entries are invalid, a class interval is invalid or intervals overlap. On successful calculation, it displays the four results and supporting details. For ungrouped data, these details include the number and sum of observations and a sorted data table. For grouped data, the details include total frequency, Σfm, the median and modal class calculations, and a frequency distribution table containing class interval, frequency, midpoint, fm and cumulative frequency.

4. Implementation

The following listings reproduce the HTML, CSS and JavaScript implementation supplied with the project. They are included as a technical record of the simulator's structure and calculation logic.

4.1 HTML

```
❖ ■Problem Statement:

Design a simulator to compute mean, mode, median and standard deviation, both for
grouped and ungrouped data

                    <!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Statistical Measures Simulator</title>

    <link rel="stylesheet" href="style1.css">
    <script src="script1.js" defer></script>
</head>

<body>

<header>
    <h1>Statistical Measures Simulator</h1>
    <p>Calculate Mean, Median, Mode and Standard Deviation</p>
</header>

<main class="container">

    <!-- Tabs -->
    <div class="tabs">

        <button class="tab active" data-type="ungrouped">
            Ungrouped Data
        </button>
```

```
    <button class="tab" data-type="grouped">
        Grouped Data
    </button>

</div>

<!-- Input Section -->
<section class="card">

    <h2 id="sectionTitle">Ungrouped Data</h2>

    <p id="instructions">
        Enter numerical values in the input table.
    </p>

    <!-- Ungrouped Input -->
    <div id="ungroupedForm">

        <h3>Ungrouped Data Input Table</h3>

        <table class="input-table">

            <thead>
                <tr>
                    <th>Serial No.</th>
                    <th>Data Value (X)</th>
                    <th>Action</th>
                </tr>
            </thead>

            <tbody id="ungroupedBody"></tbody>

        </table>

        <div class="button-group">

            <button id="addUngrouped" class="secondary">
                + Add Row
            </button>

            <button id="sampleUngrouped" class="secondary">
                Load Sample
            </button>

        </div>

    </div>

    <!-- Grouped Input -->
    <div id="groupedForm" hidden>

        <h3>Grouped Data Input Table</h3>

        <table class="input-table">

            <thead>
                <tr>
                    <th>Serial No.</th>
                    <th>Lower Limit</th>
                    <th>Upper Limit</th>
                    <th>Frequency (f)</th>
                    <th>Action</th>
                </tr>
            </thead>

            <tbody id="groupedBody"></tbody>

        </table>

        <div class="button-group">

            <button id="addGrouped" class="secondary">
                + Add Row
            </button>

            <button id="sampleGrouped" class="secondary">
                Load Sample
            </button>

        </div>

    </div>

    <button id="calculateBtn" class="primary">
        Calculate Statistics
    </button>

    <p id="error" role="alert"></p>

</section>

<!-- Results Section -->
<section class="card" id="results" hidden>

    <h2>Statistical Results</h2>

    <div class="results-grid">

        <div class="result">
```

```
                <span>Mean</span>
                <strong id="mean">—</strong>
            </div>

            <div class="result">
                <span>Median</span>
                <strong id="median">—</strong>
            </div>

             <div class="result">
                 <span>Mode</span>
                <strong id="mode">—</strong>
            </div>

            <div class="result">
                <span>Population SD</span>
                <strong id="sd">—</strong>
            </div>

        </div>

        <h3>Calculation Details</h3>

        <div id="details"></div>

    </section>

</main>

<footer>
    <p>Statistical Measures Simulator </p>
</footer>

</body>
</html>
```

4.2 CSS

```
* {
    box-sizing: border-box;
}

body {
    margin: 0;
    font-family: Arial, sans-serif;
    background: #f1f5ff;
    color: #26324a;
}

/* Header */

header {
    text-align: center;
    padding: 32px 16px;
    color: white;
    background: linear-gradient(120deg, #036685, #08a6c9);
}

header h1 {
    margin: 0 0 10px;
    font-size: clamp(25px, 4vw, 36px);
}

header p {
    margin: 0;
}

/* Main Container */

.container {
    width: min(1000px, 94%);
    margin: 28px auto;
}

/* Tabs */

.tabs {
    display: flex;
    gap: 10px;
    margin-bottom: 18px;
}

button {
    cursor: pointer;
    border: none;
    border-radius: 8px;
    padding: 12px 18px;
    font-size: 15px;
    font-weight: bold;
}

.tab {
    flex: 1;
    background: #dce4ff;
    color: #036685;
}

.tab.active {
    background: #036685;
```

```
    color: white;
}

/* Card */

.card {
    background: white;
    padding: 24px;
    margin-bottom: 22px;
    border-radius: 15px;
    box-shadow: 0 5px 20px #26324a12;
}

h2 {
    color: #036685;
    margin-top: 0;
}

/* Input Table */

.input-table {
    width: 100%;
    border-collapse: collapse;
    margin-top: 15px;

    text-align: center;
}

.input-table th {
    background: #036685;
    color: white;
    padding: 13px;
}

.input-table td {
    padding: 10px;
    border: 1px solid #dbe2ef;
}

.input-table input {
    width: 100%;
    padding: 10px;
    border: 1px solid #cbd5e1;
    border-radius: 6px;
    text-align: center;
    font-size: 15px;
}

.input-table input:focus {
    outline: 2px solid #08a6c9;
}

.delete-btn {
    background: #ffe5e5;
    color: #c62828;
    padding: 8px 12px;
}

/* Buttons */

.button-group {
    display: flex;
    gap: 10px;
    margin-top: 15px;
    flex-wrap: wrap;
}

.secondary {
    background: #e4f8f5;
    color: #087f70;
}

.primary {
    width: 100%;
    margin-top: 22px;
    background: #036685;
    color: white;
}

button:hover {
    filter: brightness(0.94);
}

/* Results */

.results-grid {
    display: grid;
    grid-template-columns: repeat(2, 1fr);
    gap: 14px;
}

.result {
    padding: 18px;
    border-radius: 10px;
    background: #eef0ff;
    border-left: 4px solid #036685;
}

.result:nth-child(2) {
```

```
    background: #e6f8f1;
    border-color: #10a878;
}

.result:nth-child(3) {
    background: #fff4df;
    border-color: #edaa26;
}

.result:nth-child(4) {
    background: #e1e6c5;
    border-color: #08a6c9;
}

.result span,
.result strong {
    display: block;
}

.result span {
    font-size: 14px;
    margin-bottom: 8px;
}

.result strong {
    font-size: 25px;
    overflow-wrap: anywhere;
}

/* Calculation Details */

#details {
    overflow-x: auto;
    line-height: 1.8;
}

#details table {
    width: 100%;
    border-collapse: collapse;
    margin-top: 12px;
}

#details th,
#details td {
    padding: 10px;
    border: 1px solid #dbe2ef;
    text-align: center;
}

#details th {
    background: #eef0ff;
}

/* Error */

#error {
    color: #c62828;
    font-weight: bold;
}

/* Footer */

footer {
    text-align: center;
    padding: 18px;
    color: #64748b;
}

/* Responsive Design */

[hidden] {
    display: none !important;
}

@media (max-width: 600px) {

    .card {
        padding: 15px;
    }

    .tabs {
        flex-direction: column;
    }

    .results-grid {
        grid-template-columns: 1fr;
    }

    .input-table {
        font-size: 12px;
    }

    .input-table th,
    .input-table td {
        padding: 6px;
    }

    .input-table input {
        padding: 7px 3px;
```

```
        font-size: 13px;
    }

    .delete-btn {
```

4.3 JavaScript

```
padding: 6px;
    }
}

/* Print */

@media print {

    .tabs,
    .button-group,
    #calculateBtn {
        display: none;
    }

    body {
        background: white;
    }

    .card {
        box-shadow: none;
    }

}

const tabs = document.querySelectorAll(".tab");

const ungroupedBody = document.getElementById("ungroupedBody");
const groupedBody = document.getElementById("groupedBody");

const ungroupedForm = document.getElementById("ungroupedForm");
const groupedForm = document.getElementById("groupedForm");

const sectionTitle = document.getElementById("sectionTitle");
const instructions = document.getElementById("instructions");

const error = document.getElementById("error");
const results = document.getElementById("results");

let currentType = "ungrouped";

// ==================================
// CREATE UNGROUPED ROW
// ==================================

function addUngroupedRow(value = "") {

    const row = ungroupedBody.insertRow();

    const serial = ungroupedBody.rows.length;

    row.innerHTML = `
        <td>${serial}</td>

        <td>
            <input type="number"
                   step="any"
                   class="data-value"
                   value="${value}"
                   placeholder="Enter value">
        </td>

        <td>
            <button class="delete-btn"
                    type="button"
                    onclick="this.closest('tr').remove(); updateSerial(ungroupedBody)">
                Remove
            </button>
        </td>
    `;

}

// ==================================
// CREATE GROUPED ROW
// ==================================

function addGroupedRow(lower = "", upper = "", frequency = "") {

    const row = groupedBody.insertRow();

    const serial = groupedBody.rows.length;

    row.innerHTML = `
        <td>${serial}</td>

        <td>
            <input type="number"
                   step="any"
                   class="lower"
                   value="${lower}"
                   placeholder="Lower">
        </td>
```

```
        <td>
            <input type="number"
                   step="any"
                   class="upper"
                   value="${upper}"
                   placeholder="Upper">
        </td>

        <td>
            <input type="number"
                   step="1"
                   min="0"
                   class="frequency"
                   value="${frequency}"
                   placeholder="Frequency">
        </td>

        <td>
            <button  class="delete-btn"
                     type="button"
                     onclick="this.closest('tr').remove(); updateSerial(groupedBody)">
                Remove
            </button>
        </td>
    `;

}

// ==================================
// UPDATE SERIAL NUMBERS
// ==================================

function updateSerial(body) {

    [...body.rows].forEach((row, index) => {
        row.cells[0].textContent = index + 1;
    });

}

// ==================================
// INITIAL ROWS
// ==================================

for (let i = 0; i < 5; i++) {
    addUngroupedRow();
}

for (let i = 0; i < 4; i++) {
    addGroupedRow();
}

// ==================================
// ADD ROW BUTTONS
// ==================================

document.getElementById("addUngrouped").onclick = () => {
    addUngroupedRow();
};

document.getElementById("addGrouped").onclick = () => {
    addGroupedRow();
};

// ==================================
// TAB SWITCHING
// ==================================

tabs.forEach(tab => {

    tab.addEventListener("click", () => {

        currentType = tab.dataset.type;

        tabs.forEach(t =>
            t.classList.toggle("active", t === tab)
        );

        ungroupedForm.hidden =
            currentType !== "ungrouped";

        groupedForm.hidden =
            currentType !== "grouped";

        sectionTitle.textContent =
            currentType === "grouped"
                ? "Grouped Data"
                : "Ungrouped Data";

        instructions.textContent =
            currentType === "grouped"
                ? "Enter class intervals and frequencies."
                : "Enter numerical values in the table.";

        error.textContent = "";

        results.hidden = true;

    });
```

```
});

// ==================================
// SAMPLE DATA
// ==================================

document.getElementById("sampleUngrouped").onclick = () => {

    ungroupedBody.innerHTML = "";

    [2, 4, 4, 5, 6, 7, 4, 8].forEach(value => {
        addUngroupedRow(value);
    });

};

document.getElementById("sampleGrouped").onclick = () => {

    groupedBody.innerHTML = "";

    const sample = [
        [0, 10, 5],
        [10, 20, 8],
        [20, 30, 12],
        [30, 40, 5]
    ];

    sample.forEach(row => {
        addGroupedRow(row[0], row[1], row[2]);
    });

};

// ==================================
// CALCULATE BUTTON
// ==================================

document.getElementById("calculateBtn").onclick = () => {

    error.textContent = "";

    results.hidden = true;

    try {

        if (currentType === "ungrouped") {
            calculateUngrouped();
        } else {
            calculateGrouped();
        }

    } catch (e) {

        error.textContent = e.message;

    }

};

// ==================================
// DISPLAY RESULTS
// ==================================

function showResults(mean, median, mode, sd, details) {

    document.getElementById("mean").textContent =
        mean.toFixed(2);

    document.getElementById("median").textContent =
        median.toFixed(2);

    document.getElementById("mode").textContent =
        mode;

    document.getElementById("sd").textContent =
        sd.toFixed(2);

    document.getElementById("details").innerHTML =
        details;

    results.hidden = false;

}

// ==================================
// UNGROUPED DATA CALCULATION
// ==================================

function calculateUngrouped() {

    const inputs = [
        ...ungroupedBody.querySelectorAll(".data-value")
    ];

    const values = inputs.map(input => {

        const text = input.value.trim();

        if (text === "") {
```

```
        throw new Error("Please fill all data values.");
    }

    return Number(text);

});

if (values.length === 0) {
    throw new Error("Please enter some data.");
}

if (values.some(v => !Number.isFinite(v))) {
    throw new Error("Please enter valid numerical values.");
}

const n = values.length;

const sorted = [...values].sort((a, b) => a - b);

// Mean

const sum = values.reduce((a, b) => a + b, 0);

const mean = sum / n;

// Median

const median = n % 2 === 1
    ? sorted[Math.floor(n / 2)]
    : (sorted[n / 2 - 1] + sorted[n / 2]) / 2;

// Mode

const counts = new Map();

values.forEach(v => {
    counts.set(v, (counts.get(v) || 0) + 1);
});

const maxFrequency = Math.max(...counts.values());

const modes = [...counts.entries()]
    .filter(([value, frequency]) =>
        frequency === maxFrequency
    )
    .map(([value]) => value);

const mode = maxFrequency === 1

    ? "No mode"
    : modes.join(", ");

// Population Variance

const variance = values.reduce(
    (total, v) => total + (v - mean) ** 2,
) / n;

// Standard Deviation

const sd = Math.sqrt(variance);

// Sorted Data Table

const rows = sorted.map((v, i) => `
    <tr>
        <td>${i + 1}</td>
        <td>${v}</td>
    </tr>
`).join("");

const details = `

    <p><b>Number of observations (n):</b> ${n}</p>

    <p><b>Sum of values (Σx):</b> ${sum}</p>

    <p><b>Mean:</b> Σx / n = ${sum} / ${n}</p>

    <p><b>Median:</b> Middle value of sorted data</p>

    <p><b>Mode:</b> Value with the highest frequency</p>

    <p><b>Population Variance:</b> ${variance.toFixed(4)}</p>

    <p><b>Standard Deviation:</b> √Variance</p>

    <h3>Sorted Data Table</h3>

    <table>

        <tr>
            <th>Position</th>
            <th>Value</th>
        </tr>

        ${rows}

    </table>
```

```
    `;

    showResults(mean, median, mode, sd, details);

}

// ================================
// GROUPED DATA CALCULATION
// ================================

function calculateGrouped() {

    const rows = [...groupedBody.rows];

    const classes = rows.map((row, index) => {

        const lowerText = row.querySelector(".lower").value.trim();
        const upperText = row.querySelector(".upper").value.trim();
        const frequencyText = row.querySelector(".frequency").value.trim();

        if (lowerText === "" || upperText === "" || frequencyText === "") {
            throw new Error(`Please fill all fields in row ${index + 1}.`);
        }

        const lower = Number(lowerText);
        const upper = Number(upperText);
        const f = Number(frequencyText);

        if (
            !Number.isFinite(lower) ||
            !Number.isFinite(upper) ||
            !Number.isFinite(f) ||
            upper <= lower ||
            f < 0 ||
            !Number.isInteger(f)
        ) {
            throw new Error(`Invalid data in row ${index + 1}.`);
        }

        return {
            lower,
            upper,
            f,
            mid: (lower + upper) / 2
        };

    });

    if (classes.length === 0) {
        throw new Error("Please enter grouped data.");
    }

    classes.sort((a, b) => a.lower - b.lower);

    // Check overlapping intervals

    for (let i = 1; i < classes.length; i++) {

        if (classes[i].lower < classes[i - 1].upper) {
            throw new Error("Class intervals must not overlap.");
        }

    }

    // Total Frequency

    const N = classes.reduce((s, c) => s + c.f, 0);

    if (N === 0) {
        throw new Error("Total frequency must be greater than zero.");
    }

    // Mean

    const sumFx = classes.reduce(
        (s, c) => s + c.f * c.mid,
    );

    const mean = sumFx / N;

    // Variance

    const variance = classes.reduce(
        (s, c) => s + c.f * (c.mid - mean) ** 2,
    ) / N;

    // Standard Deviation

    const sd = Math.sqrt(variance);

    // Median

    let cumulative = 0;
    let median = null;
    let medianClass = null;

    for (const c of classes) {

        const previous = cumulative;
```

```
    cumulative += c.f;

    if (
        median === null &&
        c.f > 0 &&
        cumulative >= N / 2
    ) {

        medianClass = {
            ...c,
            previous
        };

        median = c.lower +
            ((N / 2 - previous) / c.f) *
            (c.upper - c.lower);

    }

}

// Mode

let mode = "Not defined";

const modalIndex = classes.reduce(
    (best, c, i, arr) =>
        c.f > arr[best].f ? i : best,
);

const modal = classes[modalIndex];

if (modal.f > 0) {

    const previousF =
        modalIndex > 0
            ? classes[modalIndex - 1].f
            : 0;

    const nextF =
        modalIndex < classes.length - 1
            ? classes[modalIndex + 1].f
            : 0;

    const d1 = modal.f - previousF;
    const d2 = modal.f - nextF;

    const width = modal.upper - modal.lower;

    if (d1 + d2 !== 0) {

        mode = (
            modal.lower +
            (d1 / (d1 + d2)) * width
        ).toFixed(2);

    }

}

// Cumulative Frequency and Table

let cf = 0;

const tableRows = classes.map(c => {

    cf += c.f;

    return `
        <tr>
            <td>${c.lower} – ${c.upper}</td>
            <td>${c.f}</td>
            <td>${c.mid.toFixed(2)}</td>
            <td>${(c.f * c.mid).toFixed(2)}</td>
            <td>${cf}</td>
        </tr>
    `;

}).join("");

const details = `

    <p><b>Total Frequency (N):</b> ${N}</p>

    <p><b>Σfx:</b> ${sumFx.toFixed(2)}</p>

    <p><b>Mean:</b> Σfx / N =
        ${sumFx.toFixed(2)} / ${N}
    </p>

    <p><b>Median Formula:</b>
        L + [(N/2 - cf) / f] × h
    </p>

    <p><b>Median Class:</b>
        ${medianClass.lower} – ${medianClass.upper}
    </p>

    <p><b>Median:</b> ${median.toFixed(2)}</p>
```

5. Results

The original assignment includes screenshots of the simulator running with sample datasets. The ungrouped sample contains the values 2, 4, 4, 5, 6, 7, 4 and 8. The grouped sample contains four intervals: 0?10 (frequency 5), 10?20 (frequency 8), 20?30 (frequency 12) and 30?40 (frequency 5). The screenshots below show the interface and calculated results.



4	5	Remove
5	6	Remove
6	7	Remove
7	4	Remove
8	8	Remove

+ Add Row Load Sample

Calculate Statistics

Statistical Results

Mean
5.00

Median
4.50

Mode
4

Population SD
1.80

Calculation Details

Number of observations (n): 8

Sum of values (Σx): 40

Mean: Σx / n = 40 / 8

Median: Middle value of sorted data

Mode: Value with the highest frequency

Population Variance: 3.2500

Standard Deviation: √Variance

Sorted Data Table

Position	Value
1	2
2	4
3	4
4	4
5	5
6	6
7	7
8	8

Statistical Measures Simulator

Statistical Measures Simulator

Calculate Mean, Median, Mode and Standard Deviation

Ungrouped Data | Grouped Data

Grouped Data

Enter class intervals and frequencies.

Grouped Data Input Table

Serial No.	Lower Limit	Upper Limit	Frequency (f)	Action
1	0	10	5	Remove
2	10	20	8	Remove
3	20	30	12	Remove

4	30	40	5	Remove

+ Add Row Load Sample

Calculate Statistics

Statistical Results

Mean
20.67

Median
21.67

Mode
23.64

Population SD
9.55

Calculation Details

Total Frequency (N): 30

Σfx: 620.00

Mean: Σfx / N = 620.00 / 30

Median Formula: L + [(N/2 - cf) / f] × h

Median Class: 20 – 30

Median: 21.67

Mode Formula: $L + [(f_1 - f_0) / (2f_1 - f_0 - f_2)] \times h$

Population Variance: 91.2222

Frequency Distribution Table

Class Interval	Frequency (f)	Midpoint (x)	fx	Cumulative Frequency
0 – 10	5	5.00	25.00	5
10 – 20	8	15.00	120.00	13
20 – 30	12	25.00	300.00	25
30 – 40	5	35.00	175.00	30

Note: Grouped-data statistical measures are estimates based on class midpoints.

Statistical Measures Simulator

6. Discussion

The simulator demonstrates how web technologies can be combined to perform descriptive statistical calculations interactively. HTML supplies the structure of the interface, CSS improves its appearance and responsiveness, and JavaScript processes the input and applies the formulas. For ungrouped data, the calculations are based directly on the entered observations. For grouped data, the use of class midpoints and interpolation makes the results estimates rather than exact values.

The implementation uses population standard deviation, in which the sum of squared deviations is divided by N (or n). If the dataset is a sample and sample standard deviation is required, the denominator should instead be n ? 1 (with the

corresponding frequency-weighted sample formula for grouped data). The grouped-data estimates depend onassumptions about how observations are distributed within each class. Consequently, they may differ from calculationsperformed on the original individual observations. Special cases, including tied modal classes and modal classes at theboundaries, also require careful interpretation. These are limitations of the present implementation and should beconsidered when interpreting its output.

7.Conclusion

A browser-based Statistical Measures Simulator has been designed to calculate mean, median, mode and population standard deviation for both grouped and ungrouped data. The application combines dynamic data entry, validation, sample datasets and step-by-step result details in a single interface. It can support classroom learning by making the calculations visible and repeatable. Future improvements may include sample standard deviation, clearer handling of tied or boundary modal classes, export of results, and additional visualizations such as histograms and frequency polygons.

References

[1] B. Illowsky and S. Dean, *Introductory Statistics 2e*, OpenStax, Rice University, 2023, Chapter 2, especially Sections 2.5 and 2.7, “Measures of the Center of the Data” and “Measures of the Spread of the Data.” Available: https://openstax.org/details/books/introductory-statistics-2e

[2] Mozilla Developer Network (MDN), “HTML: HyperText Markup Language.” Available: https://developer.mozilla.org/en-US/docs/Web/HTML

[3] Mozilla Developer Network (MDN), “CSS: Cascading Style Sheets.” Available: https://developer.mozilla.org/en-US/docs/Web/CSS

[4] Mozilla Developer Network (MDN), “JavaScript.” Available: https://developer.mozilla.org/en-US/docs/Web/JavaScript

[5] NIST/SEMATECH, e-Handbook of Statistical Methods, National Institute of Standards and Technology. Available: https://www.itl.nist.gov/div898/handbook/

[6] D. Freedman, R. Pisani, and R. Purves, Statistics, 4th ed., W. W. Norton & Company, 2007.

[7] World Wide Web Consortium (W3C), ?HTML Living Standard,? WHATWG. Available: https://html.spec.whatwg.org/

[8] Ecma International, ?ECMAScript® Language Specification,? ECMA-262. Available: https://tc39.es/ecma262/

Note: The program listings and output screenshots in this paper are adapted from the project material submitted by the authors. The statistical background references are cited above.